Metodologías de Desarrollo Web

Máster de Ingeniería Web
E.T.S. de Ingeniería en Sistemas Informáticos
Universidad Politécnica de Madrid
Curso 2025-2026



Agustín Yagüe y Jennifer Pérez
Universidad Politécnica de Madrid
E.T.S. de Ingeniería en Sistemas Informáticos
Departamento de Sistemas Informáticos

CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

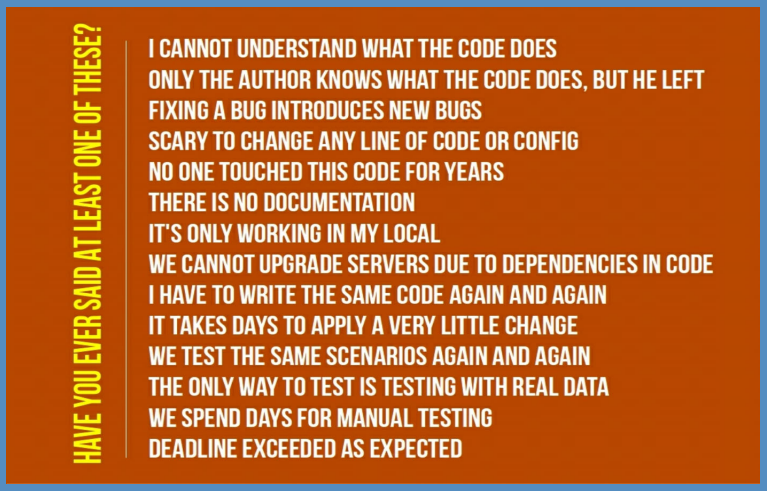
- **CONTENIDOS**
  - **Behaviour-Driven Development (BDD)**
  - **Testing - JUnit**
  - **Test-Driven Development**



Desventajas de probar después de desarrollar

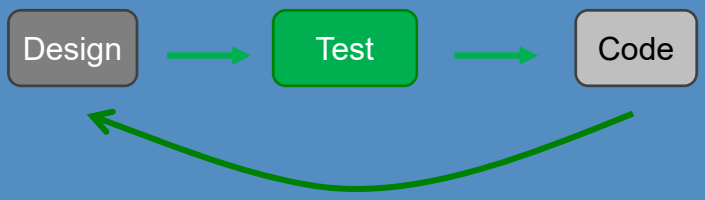
- Las pruebas no brindan retroalimentación directa al diseño y programación => en TDD, la retroalimentación se alimenta directamente volver a mejorar el diseño y los programas.
- La mayoría de las veces, después de darse cuenta de la funcionalidad en código, la prueba unitaria se omite => TDD invierte esto secuencia y ayuda a crear pruebas unitarias primero.
- Escribir pruebas después de desarrollar código a menudo resulta en pruebas de "camino feliz" => no tenemos suficiente segmentos de código granulares o "comprobables" para escribir las pruebas

TDD busca resolver …



¿Qué es TDD?

- Es una técnica para construir software en el que el desarrollo se guía por la escritura de pruebas. (Martin Fowler)
- TDD NO es sólo escribir pruebas y luego desarrollar sino que tiene que ver con diseño y con refactorización..
- Las pruebas las escribe el desarrollador no el probador



¿Qué es TDD?

- Está relacionado con el diseño
- Busca aumentar la confianza
- Facilita el cambio y la gestión del cambio
- Está relacionado con la automatización
- Permite validar el diseño realizado
- Es un caso particular de especificación mediante ejemplos
- Aporta feedback rápido
- Aumenta la calidad de la implementación y del diseño
- Fuerza a la integración continua
- Requiere mas disciplina y autodisciplina
- Genera soluciones profesionalizadas
- Los desarrolladores escriben pruebas, aunque NO es el único tipo de pruebas que tienen que hacer

Principios fundamentales

- Piensa lo que tienes que hacer, lo que estás haciendo y lo que vas a hacer
- Sigue el ciclo TDD y las tres leyes
- Nunca escribas nueva funcionalidad sin un test que falle
- Codifica en pequeños incrementos
- Haz que el sistema siempre esté funcionando
  - Comprobar que los cambios no rompen el sistema
  - Los fallos detectados deben repararse de forma inmediata

Sin embargo …

1. TDD no remplaza la arquitectura ni el diseño
2. Es necesario tener en mente un diseño previo pero … NO un diseño completo y detallado
3. Hay que diferir lo más posible las decisiones arquitectónicas

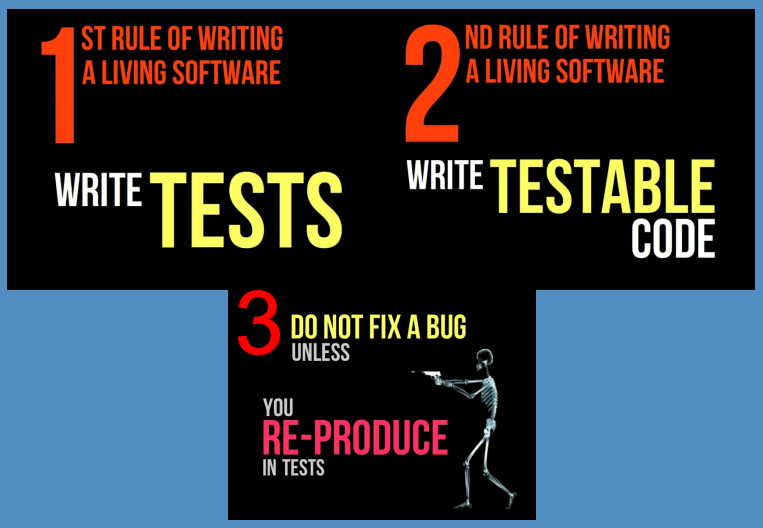
Las tres leyes de TDD

1. No puedes empezar a escribir código de la solución hasta que no has escrito una prueba que falla.

2. No debes escribir más de una prueba a la vez que haga fallar el software o producir un fallo de compilación.

3. Tienes que escribir el código necesario y suficiente para pasar el test que falla.

Reglas de TDD



Las herramientas son necesarias



Continuous Integration

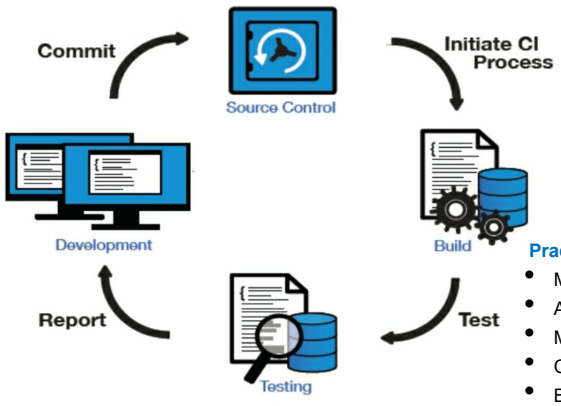
*"Continuous Integration is a software development practice where members of a team **integrate** their work **frequently**, usually each person integrates at least daily - leading to multiple integrations per day. Each integration is **verified** by an **automated** build (including test) to **detect integration errors** as quickly as possible. Many teams find that this approach leads to significantly reduced integration problems and allows a team to develop cohesive software more rapidly."*

CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

Continuous Integration steps

1. Take a copy of the source from the source code control system onto local machine
2. Implementation tasks
   1. Coding
   2. Testing
3. Automated build
   1. Compiles
   2. Link source code into an executable
   3. Run automated tests
4. If build is success then commit changes to the repository
   1. Update the working copy with external changes
   2. Rebuild
   3. Commit changes
5. Build again in the integration machine
   1. if success work done

CONTINUOUS INTEGRATION



Practice of Continuous Integration

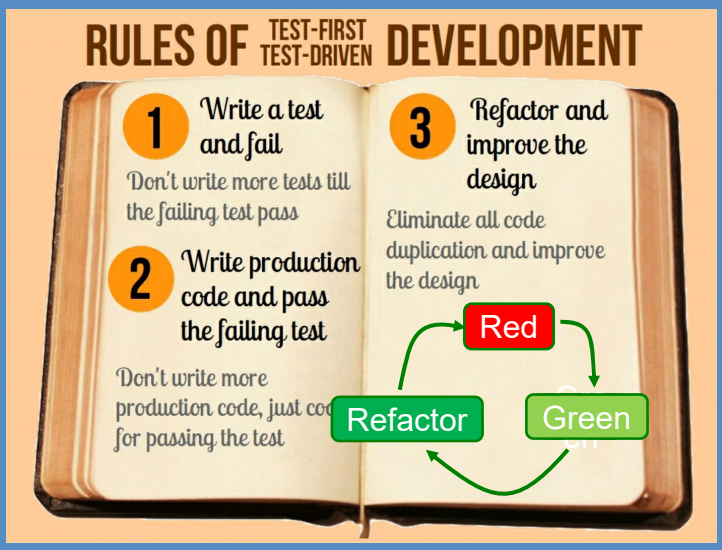
- Maintain a Single Source Repository
- Automate the build
- Make your build self-testing
- Commit changes every day
- Build on an Integration Machine
- Fix broken builds
- Automate deployment

CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

Verification tools

- Benefits of Continuous Integration
  - Short integration time
  - Bugs are detected while integrating code
  - Increases the number of tests
  - Provides support for frequent deployment
- Some tools
  - Jenkins/Hudson
  - Continuum
  - Bamboo
  - Circle CI

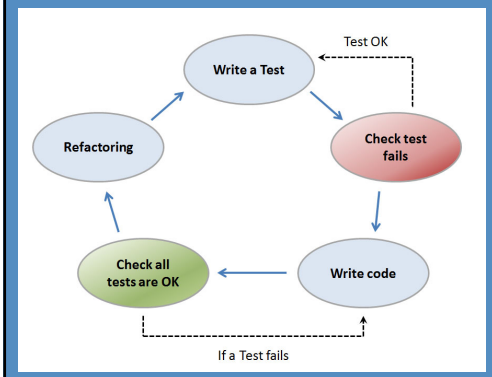
MDW: Metodologías Web



Make it green, then make it clean!

Ciclo del Proceso TDD

Make it green, then make it clean!



- **Red**: escribe una prueba que falle, incluso que no compile
- **Verde**: haz que el código pase la prueba
- **Refactorizar**: elimina el código duplicado y los "code smells"
- **Ciclo rápido**, no más de 10 minutos

Propiedades de las pruebas

FIRST

- **F**ast: a test must be fast to be executed often.
- **I**ndependent: tests must not depend on each other.
- **R**epeatable: a test must be reproducible in any environment.
- **S**elf-Validating: a test must have a binary result (Failure or Success) for a quick and easy conclusion.
- **T**imely: a test must be written at the appropriate time, i.e. just before the production code it will validate.

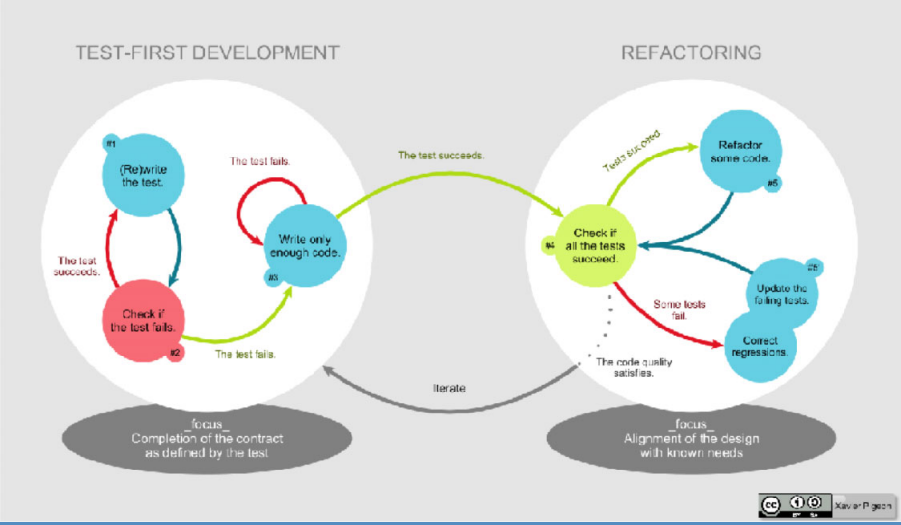
CAMPUS DE EXCELENCIA INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

Propiedades de las pruebas

Buenas practicas para escribir pruebas

- El nombre del test tiene que describir una característica o especificación
- El nombre debe de describir el propósito de la prueba y que sirva como documentación
- Cada prueba sirve para un único concepto
- Cada prueba SÓLO puede tener una aserción
- Centrarse en probar qué hace el objeto probado y no lo que es el objeto.

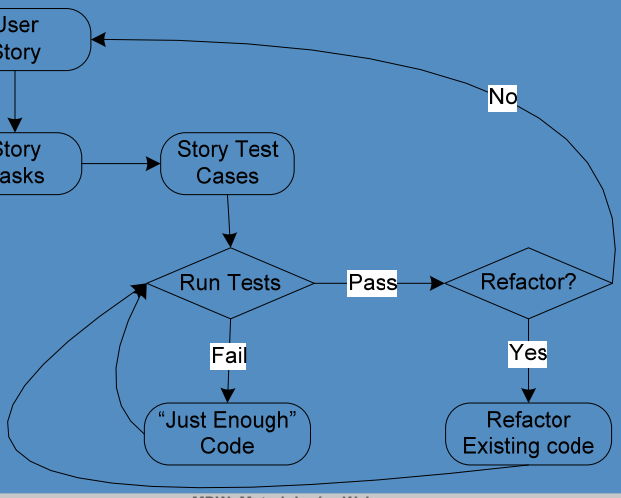
TDD y refactoring



Beneficios de TDD

Mejor diseño	Código limpio
Refactorización más segura	Incremento de la calidad
Mejor cobertura de código	Las pruebas sirven como documentos
Depuración más rápida	Mejora la detección de fallos
Pruebas autodocumentadas	Las pruebas indican cómo usar el código

TDD y las historias de usuario



Las 13 recomendaciones para hacer TDD

1. Escribir el test primero
2. Siempre se resuelve el caso más sencillo
3. Poner un único Assert en el test
4. No escribir más de un test a la vez
5. No hay que creer que TDD es únicamente usar Xunit
6. Hay que nombrar los tests por lo que hacen (qué) no por cómo lo hacen
7. Verificar una única funcionalidad por test
8. No hay que hacer dos pruebas iguales
9. Hay que mantener los tests limpios
10. No se empiezan a probar los interfaces de las clases se empieza por la lógica de negocio
11. No se usarán bases de datos mientras se usa TDD
12. No se accede a sistemas externos mientras se hace TDD
13. El rendimiento del sistema no es lo primero a probar

¡Practica!
¡Practica!
¡Practica!

Ejemplo de aplicación TDD

Aplicar TDD para determinar una función que determine si un año es bisiesto.

Un año es bisiesto si es múltiplo de 4, pero los años del siglo son bisiestos solo si son múltiplos de 400.

Aplicación de TDD

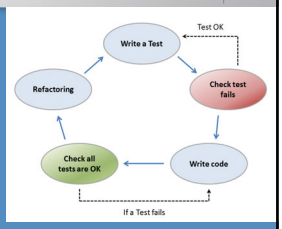
- **Paso 1**

Un año es bisiesto si es divisible por 4.



¿Alguna prueba y su valor esperado?

Aplicación de TDD

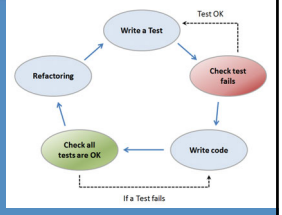


- **Paso 1**
- **Prueba 1**
  - Entrada = 2004
  - Valor esperado = Verdadero



- Ejecutar la función …. success = isLeap (2004)
- ¿Falla? -> Claro no hemos definido la función

Aplicación de TDD



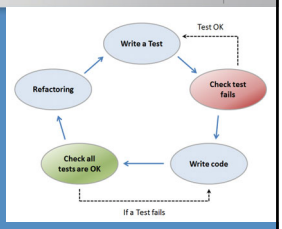
- **Paso 1**
- **Prueba 1**
  - Entrada = 2004
  - Valor esperado = Verdadero



```
Function isLeap(year) As Boolean
   dim year AS Integer
   'MOD is the modulo arithmetic built-in operator in
most languages
   If (( year MOD 4) = 0) Then
       IsLeap = True
   EndIf
End isLeap
```

- Ahora ya sí que pasa la prueba

Aplicación de TDD



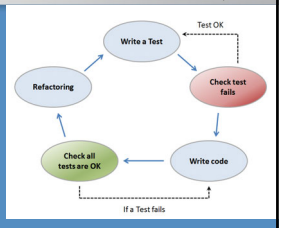
- **Prueba 1**
  - Entrada = 2004
  - Valor esperado = Verdadero
- **Prueba 2**
  - Entrada = 2007
  - Valor esperado = Falso



```
Function isLeap(year) As Boolean
    dim year AS Integer
    'MOD is the modulo arithmetic built-in operator in
most languages
    If (( year MOD 4) = 0) Then
        IsLeap = True
    EndIf
End isLeap
```

- Sigue pasando la prueba la prueba 1, pero falla en la 2

Aplicación de TDD



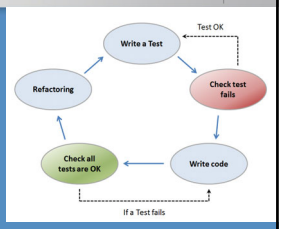
- **Prueba 1**
  - Entrada = 2004
  - Valor esperado = Verdadero
- **Prueba 2**
  - Entrada = 2007
  - Valor esperado = Falso



```
Function isLeap(year) As Boolean
   dim year AS Integer
   'MOD is the modulo arithmetic built-in operator in most languages
   If (( year MOD 4) = 0) Then
       IsLeap = True
   Else
    IsLeap = False
   EndIf
End isLeap
```

Ahora pasa las dos pruebas

Aplicación de TDD

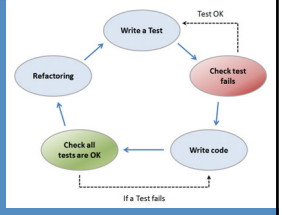


- **Refactorizar**

```
Function isLeap(year) As Boolean
   dim year AS Integer
   IsLeap = False
   'MOD is the modulo arithmetic built-in operator in most
languages
   If (( year MOD 4) = 0) Then
       IsLeap = True
   EndIf
End isLeap
```

Sigue pasando las dos pruebas

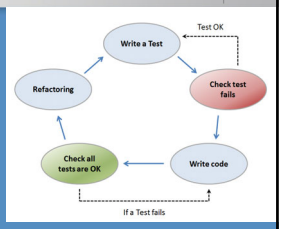
Aplicación de TDD



- **Prueba 1**
  - Entrada = 2004
  - Valor esperado = Verdadero
- **Prueba 2**
  - Entrada = 2007
  - Valor esperado = Falso
- **Prueba 3**
  - Entrada = 1900
  - Valor esperado = Falso

```
Function isLeap(year) As Boolean
dim year AS Integer
   IsLeap = False
   'MOD is the modulo arithmetic built-in operator in most languages
   If (( year MOD 4) = 0) Then
       IsLeap = True
   EndIf
End isLeap
```

Aplicación de TDD

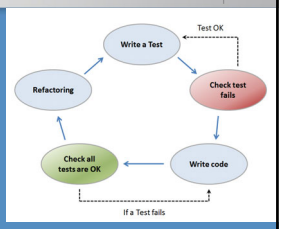


- **Prueba 1**
  - Entrada = 2004
  - Valor esperado = Verdadero
- **Prueba 2**
  - Entrada = 2007
  - Valor esperado = Falso
- **Prueba 3**
  - Entrada = 1900
  - Valor esperado = Falso



```
Function isLeap(year) As Boolean
dim year AS Integer
   IsLeap = False
   'MOD is the modulo arithmetic built-in operator in most languages
   If ((( year MOD 4) = 0) AND NOT((year MOD 100) = 0))) Then
      IsLeap = True
   EndIf
End isLeap
```


Aplicación de TDD

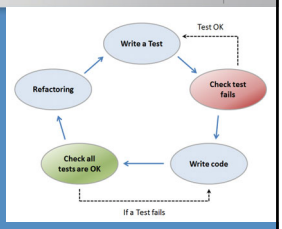


- **Prueba 1**
  - Entrada = 2004
  - Valor esperado = Verdadero
- **Prueba 2**
  - Entrada = 2007
  - Valor esperado = Falso
- **Prueba 3**
  - Entrada = 1900
  - Valor esperado = Falso
- **Prueba 4**
  - Entrada = 2000 Valor esperado = Verdadero



```
Function isLeap(year) As Boolean
dim year AS Integer
   IsLeap = False
   'MOD is the modulo arithmetic built-in operator in most languages
   If ((( year MOD 4) = 0) AND NOT((year MOD 100) = 0))) Then
      IsLeap = True
   EndIf
End isLeap
```

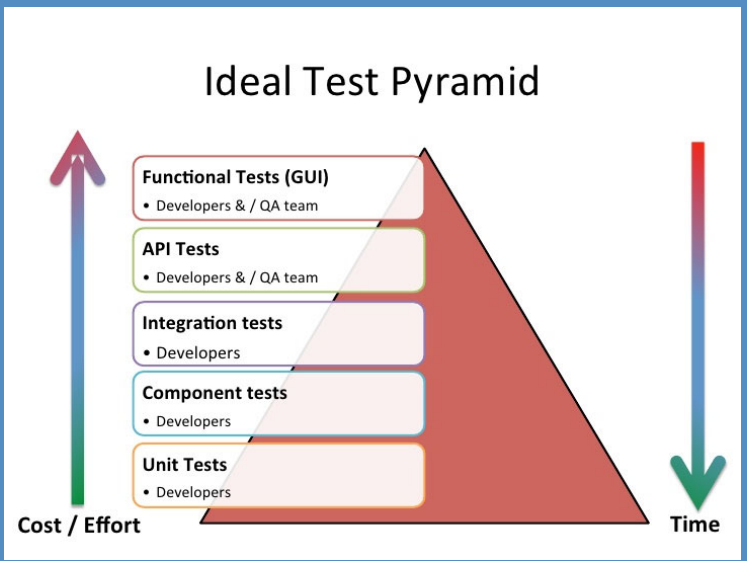
Aplicación de TDD



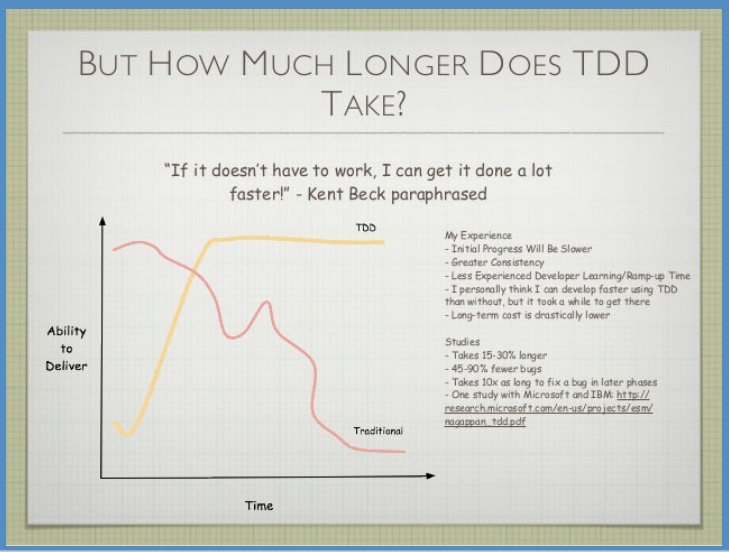
- **Prueba 1**
  - Entrada = 2004
  - Valor esperado = Verdadero
- **Prueba 2**
  - Entrada = 2007
  - Valor esperado = Falso
- **Prueba 3**
  - Entrada = 1900
  - Valor esperado = Falso
- **Prueba 4**
  - Entrada = 2000 Valor esperado = Verdadero

```
Function isLeap(year) As Boolean
dim year AS Integer
   IsLeap = False
   'MOD is the modulo arithmetic built-in operator in most languages
   if ((( year MOD 4) = 0) AND NOT((year MOD 100) = 0))) OR
    ((year MOD 400 = 0)) Then
      IsLeap = True
   EndIf
End isLeap
```


Donde encajan las pruebas en TDD



Algunas conclusiones



TDD ANTI-PATTERNS

Extraído del blog:
— *James Carr.*

THE LIAR

- EL MENTIROSO. Todos los *asserts* se cumplen pero el test no está probando la funcionalidad que realmente se quería probar.

EXCESSIVE SETUP

- EXCESIVA CONFIGURACIÓN . Demasiado código en los *Fixtures* o configuración, que impide ver lo que se está probando o la finalidad del test.

THE GIANT

- EL GIGANTE. El test es correcto pero contiene demasiado código, probando muchos casos de uso. O una clase de test con una cantidad enorme de tests. Indica necesidad de aplicar SRP.

THE MOCKERY

- EL EMULADOR. Se abusa de los mocks, stubs o fakes. El sistema no acaba siendo testeado, se están validando los datos devueltos por los mocks. Demasiada verificación por interacción. El test es un reflejo del código funcional, que tan solo prueba cómo se usan las dependencias.

TDD ANTI-PATTERNS

THE INSPECTOR

- EL INSPECTOR. Test que viola la encapsulación (partes privadas) o desvela demasiados detalles de implementación para conseguir 100% de cobertura. Cualquier refactorización posterior romperá el test.

GENEROUS LEFTOVERS

- SOBRAS ABUNDANTES. Tests que violan la propiedad de independencia. Los datos de un test se mantienen y no se limpian, aprovechándose o interfiriendo en otros tests.

THE LOCAL HERO

- EL HÉROE LOCAL. Los tests solo funcionan en una plataforma específica o con una configuración específica. No sirven para TDD, los tests se deben ejecutar en el equipo de cualquier desarrollador.

THE NITPICKER

- EL QUISQUILLOSO. El test valida la salida completa o demasiados detalles no significativos. Cada pequeño cambio requiere el mantenimiento del test. (Ej. Comparar el title o el css de las páginas html de una aplicación web).

TDD ANTI-PATTERNS

THE SECRET CATCHER

- CAZADOR SECRETO. Parece que no realiza verificaciones por ausencia de *asserts*, pero en realidad verifica que no se lanza alguna excepción. Conviene aclarar la finalidad capturando la excepción y finalizar con un *fail.*

THE DODGER

- EL DESERTOR. Test que prueba muchos efectos colaterales, pero no termina de probar la autentica funcionalidad el *Object Under Test.* Ej. Verificar que no se alteran datos, o no se ejecutan ciertos métodos de las dependencias, etc.

THE LOUDMOUTH

- EL ALBOROTADOR. Llena la consola de demasiados datos de diagnostico, trazas, mensajes, etc. incluso cuando se pasan los tests. Algunas veces se crean estas trazas durante la codificación de los test, y al final se dejan aunque no sean necesarios (enturbiando los resultados).

THE GREEDY CATCHER

- CAZADOR GLOTÓN. Captura e ignora las excepciones (aunque escriba trazas), dejando pasar los tests. Problemática muy habitual en aplicaciones Java (incluso en código funcional).

TDD ANTI-PATTERNS

THE SEQUENCER

- EL SECUENCIADOR. Fija el orden de los resultados. Los asserts esperan que los resultados se reciban siempre en el mismo orden (métodos que devuelven listas).

HIDDEN DEPENDENCY

- DEPENDENCIA OCULTA. Test que presupone la existencia de ciertos datos de test en la plataforma de ejecución. Otros desarrolladores deben descubrir que datos se supone que deberían existir.

THE ENUMERATOR

- EL ENUMERADOR. Los nombres de los métodos de test son sólo una enumeración como test1, test2, test3, en lugar de utilizar nombres significativos que indiquen la intención de cada uno de los tests.

THE STRANGER

- EL EXTRAÑO. Test que está realizando pruebas sobre un objeto que no pertenece al *Object Under Test* de la clase de test. Una clase de test se debería encargar de los test de una sola clase del código funcional. También *El Pariente Lejano (Distant Relative)*.

TDD ANTI-PATTERNS

THE OPERATING SYSTEM EVANGELIST

- EVANGELISTA DEL SISTEMA OPERATIVO. Los test que dependen de caracteristicas del sistema operativo. Ej. Suposiciones de los caracteres de nueva línea, comandos de la shell, etc.

SUCCESS AGAINST ALL ODDS

- A PESAR DE TODO FUNCIONA. Se escribe el código funcional antes de hacer fallar al test aunque este se escribiera antes, o se escriben test sobre código que ya existe. Un efecto lateral es que el test puede pasar aunque el código tenga fallos.

THE FREE RIDE

- DOS POR UNO. En lugar de escribir un nuevo test para probar una nueva funcionalidad, se añade un nuevo assert en un test ya existente.

THE ONE

- EL ÚNICO. Combinación de varios patrones (*Free ride and Giant*). Un solo método de test que prueba todos los casos de uso del objeto en secuencia. Contiene múltiples fixtures y asserts.

TDD ANTI-PATTERNS

THE PEEPING TOM

- EL MIRÓN. Debido a recursos compartidos un test puede ver los resultados de otro test y puede hacer que el test falle incluso cuando el sistema testeado es válido. Suele producirse con el uso de variables de clase (miembros estáticos). También conocido como *The Uninvited Guests*.

THE SLOW POKE

- EL TORTUGA. Un test increíblemente lento. No apto para el desarrollo con TDD, si no para tests ejecutados en la herramienta de integración continua.

POLITÉCNICA "Ingeniamos el futuro"
CAMPUS DE EXCELENCIA INTERNACIONAL

Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

Metodologías de Desarrollo Web

Máster de Ingeniería Web
E.T.S. de Ingeniería en Sistemas Informáticos
Universidad Politécnica de Madrid
Curso 2025-2026



Agustín Yagüe y Jennifer Pérez
Universidad Politécnica de Madrid
E.T.S. de Ingeniería en Sistemas Informáticos
Departamento de Sistemas Informáticos